

Primary Key Constraint in SQL Server with Example

In this article, I am going to discuss **Primary Key Constraint in SQL Server** with Examples. Please read our previous article where we discussed the basics of [SQL Server Constraint](#). In our previous article, we discussed **Default, Not Null, Unique and Check Constraint in SQL Server** with some examples. As part of this article, we are going to discuss the following pointers.

1. **What is Primary Key in SQL Server?**
2. **Multiple Examples to understand the need for Primary Key.**
3. **What is a Composite Primary key in SQL Server?**
4. **What are the differences between the primary key and unique key in SQL Server?**

What is Primary Key in SQL Server?

The **Primary Key in SQL Server** is the combination of **Unique** and **Not Null** Constraint. That means it will not allow either **NULL** or **Duplicate** values into a column or columns on which the primary key constraint is applied. Using the primary key we can enforce entity integrity i.e. using the primary key value we should uniquely identify a record.

A table should contain only 1 Primary Key which can be either on a single or multiple columns i.e. the composite primary key. A table should have a primary key to uniquely identify each record. The Primary Key constraint can be applied to any data type like integer, character, decimal, money, etc.

Understanding the Primary Key Constraint in SQL Server:

We cannot apply more than one primary key in a table. Let us prove this. Please try to execute the below SQL Script where we used the Primary key on more than one column (Bcode and Blocation).

```
CREATE TABLE Branches
(
    Bcode INT PRIMARY KEY,
    Bname VARCHAR(40),
    Bloc CHAR(40) PRIMARY KEY
)
```

When we try to execute the above query it will give us the error as: **“Cannot add multiple PRIMARY KEY constraints to table ‘Branches’.”**

To overcome the above drawback rewrite the query like below

```
CREATE TABLE Branches
(
    Bcode INT PRIMARY KEY,
    Bname VARCHAR(40),
    Bloc CHAR(40)
)
```

Now when we execute the above code it successfully executed and create the Branches table. Here, we marked Bcode column as the primary key. Let's execute the following insert statement to insert a record into

the Branches table

INSERT Branches VALUES (1021, 'SBI', 'SRNAGAR') — It executed successfully.

Adding Duplicate value in the Primary Key column:

Try to insert a duplicate value into the Primary key column. Please execute the insert statement to insert a duplicate record and see what happens.

INSERT Branches VALUES (1021,'SBI','SRNAGAR') — Not Allowed

When we execute the same insert statement it will give us the following error. That means the Primary Key constraint will not accept duplicate values in it.

Violation of PRIMARY KEY constraint 'PK__Branches__8880509B1333792F'. Cannot insert duplicate key in object 'dbo.Banches'.
The duplicate key value is (1021).

The statement has been terminated.

Inserting NULL in Primary Key Column in SQL Server:

Let us try to insert NULL into the primary key column by executing the below insert statement and see what happens.

INSERT Branches VALUES (NULL, 'SBI', 'SRNAGAR')

When we execute the above insert statement it gives us the following error stating cannot insert null value which proves that Primary Key will not accept NULL.

Cannot insert the value NULL into column 'Bcode', table 'TestDB.dbo.Banches'; column does not allow nulls. INSERT fails.

The statement has been terminated.

What is the Composite Primary key in SQL Server?

It is also possible in SQL Server to create the Primary Key constraint on more than one columns and when we do so, it is called a Composite Primary Key. The maximum number of columns including in the composite primary key is 16 columns. It is only possible to impose the Composite Primary Key at the table level, it is not possible at the column level.

Note: In a composite primary key, each column can accept duplicate values but the duplicate combination should not be duplicated.

Let us understand this with an example.

Please have a look at the following create table statement. Here, we impose the Primary Key Constraint at the table level i.e. after all the columns are created. While creating the primary key we are providing two

columns i.e. City and Bcode. As the Primary Key is created based on more than one column, it is termed as a composite primary key.

```
CREATE TABLE BranchDetails
(
    City VARCHAR(40),
    Bcode INT,
    Bloc VARCHAR(30),
    PRIMARY KEY(City, Bcode)
)
```

Please execute the following insert statement to insert a record into the BranchDetails table.

Insert into BranchDetails (City, Bcode, Bloc) values('Mumbai', 10, 'Goregaon')

The above insert statement executed successfully. Let us try to execute the below Insert statement and see what happens.

Insert into BranchDetails (City, Bcode, Bloc) values('Mumbai', 10, 'Malad')

When we execute the above insert query it will display the following error i.e. cannot insert duplicate value.

Violation of PRIMARY KEY constraint 'PK__BranchDe__064CA565A64FECC6'. Cannot insert duplicate key in object 'dbo.BranchDetails'. The duplicate key value is (Mumbai, 10).

The statement has been terminated.

Let try to execute the following two statements.

Insert into BranchDetails (City, Bcode, Bloc) values('Mumbai', 20, 'Malad')
Insert into BranchDetails (City, Bcode, Bloc) values('Hyderabad', 20, 'SR Nagar')

The above two statements are executed successfully in spite of the Bcode value is the same. This is because it will check only the combination of City and Bcode and here the combination is not duplicate.

Now try to execute the following insert statement.

Insert into BranchDetails (City, Bcode, Bloc) values('Hyderabad', 20, 'Begumpet')

When we execute the above insert query it will display the following error. This is because the combination of the City and Bcode value already exists in the database.

Violation of PRIMARY KEY constraint 'PK__BranchDe__064CA565A64FECC6'. Cannot insert duplicate key in object 'dbo.BranchDetails'. The duplicate key value is (Hyderabad, 20).

The statement has been terminated.

Note: The primary key is also called a candidate key.

When do we need to choose Primary Key?

When we need the following features on a column, then we need to make that column as the Primary Key

1. NULLs should not be allowed.
2. It should be unique
3. It can not be modified.

What are the differences between the primary key and unique key in SQL Server?

As we already discussed both unique key and Primary Key handles the uniqueness of a column on which they are applied. But the difference is that by default in SQL Server the primary key creates a unique clustered index whereas the unique key creates a unique non-clustered index on the column. Another difference between them is that the primary key column doesn't allow NULL values whereas the unique key column allows only one NULL value.

The Primary Key Constraint in SQL Server is also used to make a relationship with a Foreign Key constraint on another table. We will discuss this in the next article. In the next article, I am going to discuss [Foreign Key Constraint in SQL Server](#) with examples. Here, in this article, I try to explain the **SQL Server Primary Key** with Examples. I hope you enjoy this Primary Key Constraint in SQL Server with Examples article.